Ex No : 3 Date: 30/07/26	Classification with Decision Trees

AIM:

To implement a Decision Tree Classifier using the Iris dataset and evaluate its performance using Accuracy Score and Confusion Matrix.

ALGORITHM:

1. Import the necessary Python libraries for data loading, model building, evaluation, and visualization.
2. Load the Iris dataset from the sklearn.datasets module.
3. Separate the dataset into input features (X) and target labels (y).
4. Split the dataset into training and testing sets using train_test_split().
5. Create a Decision Tree Classifier using the **Gini Index** as the splitting criterion.
6. Set a random state to ensure reproducible results.
7. Train the Decision Tree model using the training dataset.
8. Use the trained model to predict the class labels for the test dataset.
9. Generate the Confusion Matrix to compare the predicted labels with the actual labels.
10. Calculate the Accuracy Score to measure the overall performance of the classifier.
11. Display the Confusion Matrix using a heatmap for better visualization.
12. Visualize the trained Decision Tree to understand the decision-making process.
13. Analyze the results and verify the classification performance of the model.
14. Conclude the experiment based on the obtained accuracy and visualization results.

CODE:

```python
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Split the Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 4: Train the Decision Tree Classifier
dt_model = DecisionTreeClassifier(
    criterion='gini',
    random_state=0
)
dt_model.fit(X_train, y_train)

# Step 5: Predict
y_pred = dt_model.predict(X_test)

# Step 6: Evaluate the Model
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)

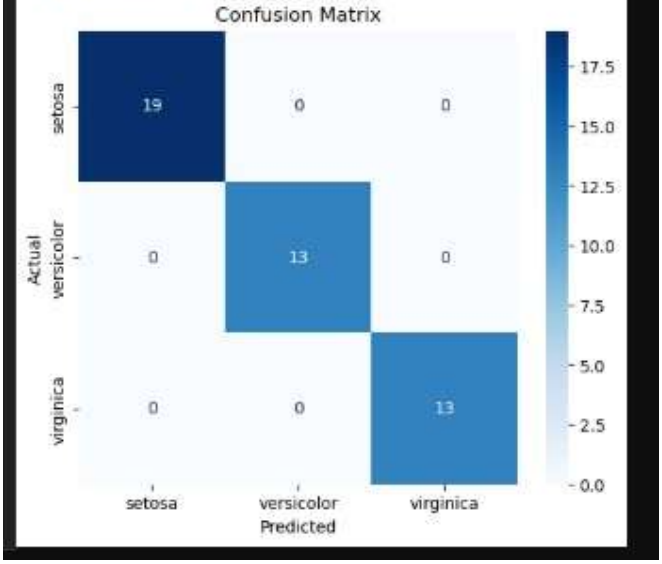
print("Confusion Matrix:\n", cm)
print("Accuracy Score:", acc)

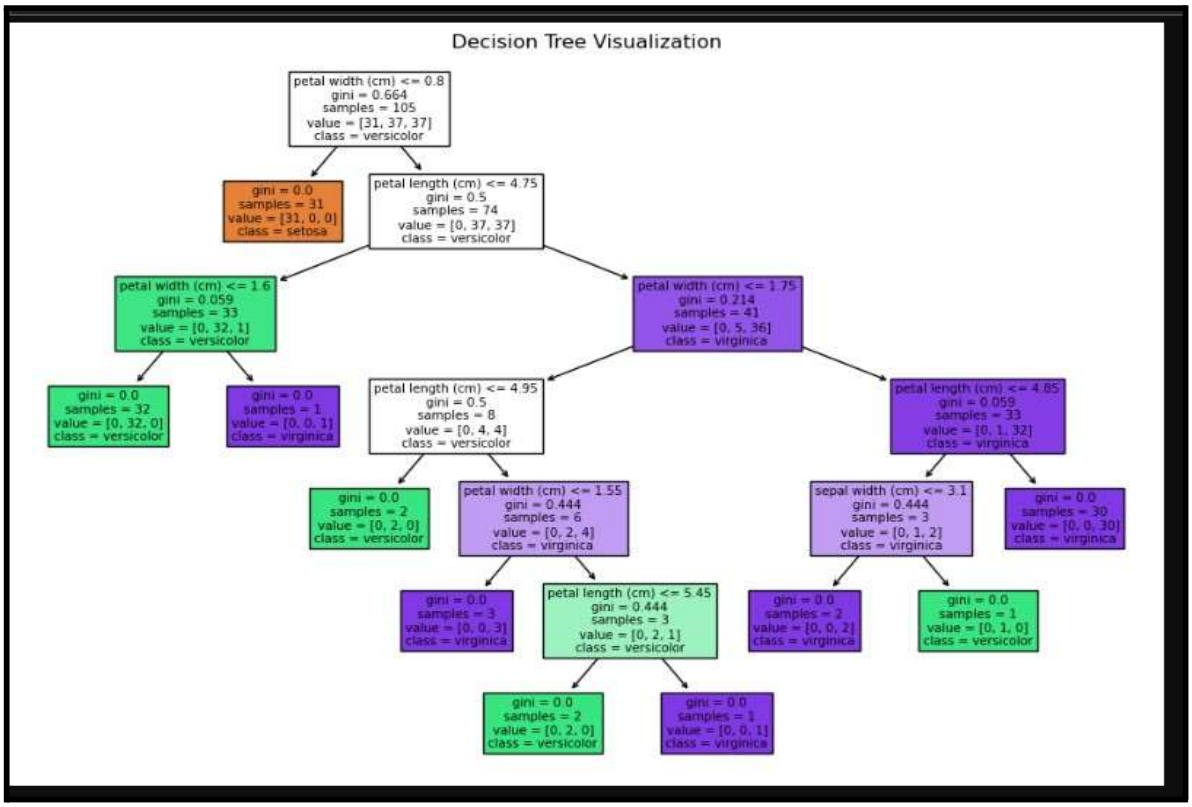
# Step 7: Visualize Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    cmap="Blues",
    fmt="d",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```python
# Step 8: Visualize the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(
    dt_model,
    filled=True,
    feature_names=iris.feature_names,
    class_names=iris.target_names
)
plt.title("Decision Tree Visualization")
plt.show()
```

OUTPUT:

```
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Accuracy Score: 1.0
```





RESULT:

The Python environment was successfully set up and the dataset was pre-processed by handling missing values, encoding categorical data, performing feature scaling, and splitting the data into training and testing sets. The dataset is now ready for model training and analysis.